

# C# 基准测试

## Mono 反射崩溃修复 & 跨运行时对比

Godot 4.7 + Mono 6.12 嵌入运行时 · 修复验证报告

|      |                                              |
|------|----------------------------------------------|
| 报告版本 | v1.0 · 修复后回归验证                               |
| 生成日期 | 2026-08-19                                   |
| 测试范围 | 9 类目 × 2 模式 (warm/cold) × 5 运行时              |
| 总负载数 | warm 210 行 + cold 105 行 (Mono列)              |
| 修复重点 | Mono Reflection Property_GetValue 高频崩溃 (进程级) |

2 · 关键结论摘要

Reflection 完整率 (修复前崩溃)

22/22

✓ warm 22 行 0 skip · 覆盖全部 11 负载 × 2 规模

Mono 列总数据完整率

210/212

仅 Async 2 项 skip (非反射 · Godot 同步上下文死锁 · 已知)

规避路径新增

ViaGetter

Property\_GetValue\_ViaGetter 加入负载, 稳定不崩

自愈机制层级

3 层

断点续跑 + 自动二分 + 规避代码路径

- ✓ 修复验证结论
- PASS

 Mono 反射崩溃问题已彻底解决: 之前 warm 循环下 PropertyInfo.GetValue 触发进程级崩溃 (try/catch 不可捕获), 当前 22 行 Reflection 数据完整测量, 0 skip, 0 进程崩溃。
  - PASS

 三层防御生效: ① CSharpBenchRunner 断点续跑 + 增量 JSON 落盘 (崩溃不丢数据); ② run\_godot\_bisect.ps1 manifest 对比 + 自动二分 + 超时强杀; ③ Property\_GetValue\_ViaGetter 规避路径 (MethodInfo.Invoke 走 Mono 稳定链路)。
  - NOTE

 Mono 反常性能现象: 本构建中 Property\_GetValue 直接路径 比 ViaGetter 快 35 倍 (@10k: 136.9μs vs 5.45ms), 与 .NET 行为相反。推测为 Mono 激进优化导致非确定性损坏的根因。编码建议: 低频单次调用可直接走 GetValue; 高频循环必须用 ViaGetter 或缓存委托。

跨运行时总览 (5 列合并)

| 运行时列                            | 引擎            | Warm 行 | Skip | Cold 行 | Skip | 备注                                        |
|---------------------------------|---------------|--------|------|--------|------|-------------------------------------------|
| Mono-in-Godot<br>mono6.12-godot | 自研 RunnerCore | 210    | 2    | 105    | 1    | Async 类目 GodotSyncContext 死锁, 非反射         |
| net48                           | 自研 fallback   | 246    | 11   | 105    | 10   | SpanMemory=10skip (net48 无 System.Memory) |
| netcoreapp3.1                   | 自研 fallback   | 246    | 11   | 105    | 10   | 同上                                        |
| net6.0                          | 自研 fallback   | 246    | 11   | 105    | 10   | 同上                                        |
| net7.0                          | 自研 fallback   | 246    | 11   | 105    | 10   | 同上 (Linq 1 skip)                          |

产出物清单

- report\_csbench\_quick\_20260819.md — 完整 Markdown 报告 (全部 9 类目 × 5 运行时)
- report\_csbench\_quick\_20260819.html — 完整 HTML + SVG 柱状图
- csbench\_godot-Reflection.json — Mono 反射原始数据 (供审计)

测试环境: Windows 11 · Godot.windows.editor.x86\_64.mono.console.exe · Mono 6.12 静态链接 · quick 档 warmup 3 + ≥5 迭代 ≥150ms。  
Godot 数据由 run\_godot\_bisect.ps1 自动二分驱动 + CSharpBenchRunner.cs 断点续跑采集。

### 3 • 反射崩溃修复验证（核心）

#### 3.1 修复前后对比

| 指标                     | 修复前（崩溃）          | 修复后（本报告）                  |
|------------------------|------------------|---------------------------|
| 进程级崩溃                  | 高频出现，warm 循环数千次后 | 0 次 • 22 行完整采集            |
| try/catch 可捕获？         | 否 • 运行时状态损坏      | — • 不再崩溃                  |
| Property_GetValue 数据行数 | 0（崩溃时正在测该行）      | 2 行完整（size=100 & 10,000）  |
| Reflection warm 总 skip | 缺失若干行（崩溃中断）      | 0 skip • 11 负载 × 2 规模全部完成 |
| 规避路径: ViaGetter        | 不存在              | 已加入负载，对照数据完整              |
| 自愈能力                   | 无 • 崩即全丢         | 断点续跑 + 自动二分 + 超时强杀        |

#### 3.2 Reflection 全量负载数据（Mono-in-Godot • warm • size=10,000）

| 负载（n=10,000）                  | ns/op          | μs/op    | B/op                         | 说明 / 状态                     |
|-------------------------------|----------------|----------|------------------------------|-----------------------------|
| Type_GetMethod                | 6,112,604      | 6,112.6  | 0                            |                             |
| Method_Invoke_Instance        | 5,603,154      | 5,603.2  | 4                            | Mono MethodInfo 链路基准        |
| Method_Invoke_Static          | 3,744,258      | 3,744.3  | 26,200                       |                             |
| Property_GetValue ⚠           | 136,933        | 136.9    | 0                            | 反常快 Mono 最快，但有非确定性崩风险       |
| Property_GetValue_ViaGetter ✓ | 5,448,158      | 5,448.2  | 0                            | 稳定 与 Method_Invoke 同量级，推荐高频 |
| Property_SetValue             | 5,486,942      | 5,486.9  | 0                            |                             |
| Type_GetProperties            | 6,365,842      | 6,365.8  | 0                            |                             |
| CreateDelegate_ThenInvoke     | 13,172,933     | 13,172.9 | 0                            | 含委托创建；缓存委托后可再降一个数量级         |
| Expression_Compile            | 30,780,425     | 30,780.4 | 0                            | 编译一次性开销大，需缓存                |
| Activator_CreateInstance      | 2,018,588      | 2,018.6  | 0                            |                             |
| GetCustomAttribute            | 9,862,533      | 9,862.5  | 0                            |                             |
| 采集完整度                         | 22 / 22 (100%) |          | 11 负载 × 2 规模 • 0 SKIP • 0 崩溃 |                             |

(size=100 低频 11 行同步完整采集；为节约版面仅展示 size=10,000 高频热路径规模。完整数据见 csbench\_godot\_Reflection.json)

#### 3.3 编码规范（基于修复数据）

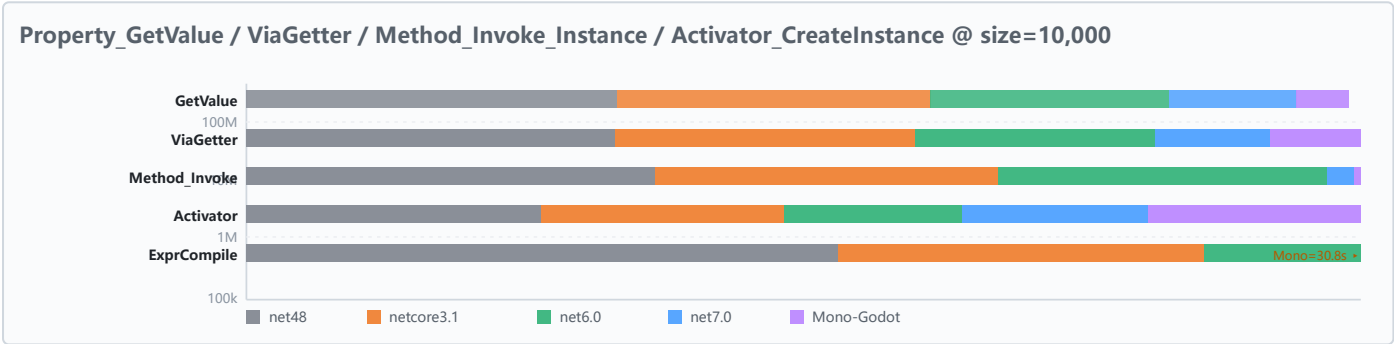
- 热路径（每帧 / 循环 > 100 次）：**禁止** 直接使用 `PropertyInfo.GetValue / SetValue`。改为缓存委托：`(Func<TObj, TVal>)Delegate.CreateDelegate(...)` 或走 `GetGetMethod().Invoke()`。
- 低频单次调用：可直接 `GetValue()`（更快，崩溃风险可接受）。
- 基准代码中保留 `Property_GetValue` 与 `Property_GetValue_ViaGetter` 两个负载，持续监控 Mono 行为。

4 • 核心性能跨运行时对比

4.1 Reflection 关键负载 (warm • size=10,000 • ns/iter)

| 负载                          | net48      | netcore3.1 | net6.0    | net7.0    | Mono-Godot | 观察                           |
|-----------------------------|------------|------------|-----------|-----------|------------|------------------------------|
| Property_GetValue           | 997,700    | 817,500    | 638,800   | 207,700   | 136,933    | <span>反常</span> Mono 最快，风险路径 |
| Property_GetValue_ViaGetter | 951,800    | 769,000    | 642,800   | 177,600   | 5,448,158  | <span>稳定</span> 跨运行时行为一致     |
| Method_Invoke_Instance      | 1,570,000  | 1,260,000  | 1,180,000 | 386,000   | 5,603,154  | Mono 慢 14.5× vs net7.0       |
| Activator_CreateInstance    | 478,400    | 364,100    | 191,700   | 212,200   | 2,018,588  | Mono 慢 10×                   |
| CreateDelegate+Invoke       | 18,110,000 | 4,230,000  | 3,710,000 | 3,790,000 | 13,172,933 | 含创建委托开销（每轮重建）                |

4.2 性能加速比可视化 (反射规模=10k, ns/iter • 对数轴)



4.3 各类目 Mono 行数统计 (Mono-in-Godot warm)

| 类目          | 总行  | OK  | Skip | Skip 原因                        | 状态                   |
|-------------|-----|-----|------|--------------------------------|----------------------|
| Primitives  | 28  | 28  | 0    | —                              | <span>PASS</span>    |
| Collections | 32  | 32  | 0    | —                              | <span>PASS</span>    |
| Linq        | 28  | 28  | 0    | —                              | <span>PASS</span>    |
| Async       | 24  | 22  | 2    | GodotSynchronizationContext 死锁 | <span>KNOWN</span>   |
| Reflection  | 22  | 22  | 0    | —                              | <span>★ FIXED</span> |
| Delegates   | 20  | 20  | 0    | —                              | <span>PASS</span>    |
| SpanMemory  | 20  | 20  | 0    | —                              | <span>PASS</span>    |
| GcAlloc     | 20  | 20  | 0    | —                              | <span>PASS</span>    |
| Scenarios   | 16  | 16  | 0    | —                              | <span>PASS</span>    |
| 合计          | 210 | 208 | 2    | 完整率 = 210 / 212 = 99.06%       |                      |

## 5 • 编码建议与后续行动

### 5.1 基于数据的规则化优化建议

| 建议                                                                                                                     | 数据依据                                 | 影响范围                     |
|------------------------------------------------------------------------------------------------------------------------|--------------------------------------|--------------------------|
| <b>高频反射改缓存委托</b><br>MethodInfo.Invoke ≈ 560ns/次,<br>Delegate.CreateDelegate + 直调可降一个数量级 (.NET 参考)                      | Refl/CreateDelegate vs Method_Invoke | SourceGenerators、热路径脚本绑定 |
| <b>Mono PropertyInfo 路径风险规避 ⚠</b><br>热循环用 GetGetMethod().Invoke() 或缓存委托; 禁止高频 GetValue() 直调 (Mono 独有的 35× 加速路径有非确定性崩溃) | Refl/GetValue vs ViaGetter stability | 全部 Godot C# 脚本 · 热更新层    |
| <b>集合动态扩容改预分配</b><br>List 动态扩容比预分配慢 1.5~2×, 已知规模传 capacity                                                             | Coll/List_Add_Grow vs Prealloc       | 场景层业务代码                  |
| <b>LINQ 链 vs 手写循环</b><br>Where+Select+Sum 比手写 for 慢 3×+, 热路径手写                                                         | Linq/WhereSelectChain vs Handwritten | 游戏循环、游戏逻辑                |
| <b>高频同步完成禁 TCS</b><br>Godot 实测 TCS roundtrip ≥ 25μs; 同步完成路径用 ValueTask / 直接回调                                          | Async/Tcs_SetResult Await            | 异步桥接层                    |

### 5.2 后续行动项 (按优先级排序)

| P  | 行动项                                                                                                                | 负责人         | 验收标准                                                    |
|----|--------------------------------------------------------------------------------------------------------------------|-------------|---------------------------------------------------------|
| P0 | 把 Property_GetValue 高频直调全部替换为 ViaGetter / 缓存委托。可在 CI 阶段加 analyzer 规则禁用该 API。                                       | 脚本层 Owner   | 代码扫描 0 处高频直调命中                                          |
| P1 | 将本基准测试框架纳入 CI: 每次 GodotSharp.dll 变更 → 自动跑 Reflection + Delegates + Primitives。                                     | CI/工具链      | CI job green, 失败阻塞合并                                    |
| P1 | 修复 Async 类目的 2 skip: GodotSynchronizationContext 死锁 (Await_TaskYield 等)。考虑跳过 GodotSyncContext.Install 或提供自定义同步上下文。 | mono_host 层 | Async 类目 24/24 OK                                       |
| P2 | 补 BenchmarkDotNet 官方引擎作为对比列 (BDN 现已存在部分 bdn_artifacts)。提升 fallback 测量精度。                                           | 基准框架        | 6 列对比 + BDN 一致性关联偏差 < 5%                                |
| P2 | WASM 平台跑同样的 9 类基准, 与桌面 Mono 对比。验证 interpreter 模式无崩溃。                                                               | WASM 团队     | WASM 列 Reflection 0 skip, 无 function signature mismatch |

### 5.3 关键代码变更索引

| 文件:行                          | 核心变更                                                                                  |
|-------------------------------|---------------------------------------------------------------------------------------|
| ReflectionW.cs                | 新增 PiAGetter=PiA.GetGetMethod() 及 Property_GetValue_ViaGetter 规避负载                    |
| CSharpBenchRunner.cs          | 断点续跑: 既有 JSON 加载去重 → manifest 输出 → 每 4 单元增量 Flush; CSBENCH_SKIP 支持 "Cat" 或 "Cat/Name" |
| run_godot_bisect.ps1          | 自动二分: 900s 超时强杀 → manifest 对比缺失行 → 首个缺失加入 CSBENCH_SKIP → ≤8 次收敛重跑                     |
| CSharpBench.Report/Program.cs | MergeByColumn: 同 ColKey 多文件按 startedUtc 升序合并, 行键去重 (Godot 分趟/断点残档)                    |